

SkillSwap – Detailed Technical Plan (Web & Mobile Application)

1. Introduction and Scope

SkillSwap is a marketplace for skill and service exchange. Users can list the skills they offer and request services they need. Exchanges are non-monetary, but the platform generates revenue through **subscriptions** and donations. The platform comprises a **website** and a **mobile application** (Android & iOS). This plan outlines the technical architecture, technology stack, data model, third-party integrations, payment processing, identity verification, and a **1-3 month** development timeline. It addresses the additional features requested by the founders (Arabic language support, mandatory memberships for listings, wish/donation workflow, banned services, ID verification, etc.) and considers high-load scenarios.

Key goals:

- Provide a **reliable, scalable and secure** backend that can handle high traffic and real-time interactions.
- Deliver a **user-friendly frontend** that supports **English and Arabic (RTL)** with responsive layouts for desktop and mobile.
- Support **free sign-up**, but require a **subscription membership** to create/accept listings. Guests can browse listings and leave reviews without paying.
- Enable a **wish/donation system** where anyone can donate anonymously without registering, but making a wish requires membership.
- Integrate a **KYC/identity verification solution** to match national ID names and prevent fraud.
- Use a **cost-effective stack** for the development phase (prefer free tiers where possible) and plan for seamless scaling in production.

2. High-Level Architecture

2.1 Frontend Architecture

Layer	Technology	Description
Web front-end	React (Next.js)	Next.js allows server-side rendering (SSR) for SEO and fast page loads and supports internationalization. React's component model improves maintainability. We will customise the existing Firebase-generated UI and replace placeholders.
Mobile app	Flutter or React Native	To reach both Android and iOS with a single codebase. Flutter offers excellent

Layer	Technology	Description
		performance and supports RTL out of the box, while React Native shares code with the web (if using React) but may require more native modules. Either option can integrate with Firebase and Geidea SDK.
Internationalization (i18n)	react-i18next / flutter_localizations	Provide translations for English and Arabic. Use the dir="rtl" attribute for Arabic to mirror layouts. A blog on RTL support notes that RTL requires adjusting text alignment, content direction, scrollbar placement and mirroring icons; using the dir="rtl" attribute on the root element automatically handles many of these issues. CSS logical properties (margin-inline-start, padding-inline-end etc.) help handle both directions.
Component library	Tailwind CSS (with RTL plugin) or Material UI	Use a design system that supports RTL; Tailwind has plugins for RTL. Material UI offers built-in RTL support.
State management	React Query / Redux Toolkit	Manage API calls, caching and global state such as user session, language selection and subscription status.
Routing	Next.js routing for the web, and React Navigation/Flutter Navigator for the app.	

2.2 Backend Architecture

We propose a **layered, microservice-friendly architecture** hosted in the cloud. The key components are:

1. **API gateway & authentication layer** – Provides REST/GraphQL endpoints and enforces authentication and rate limiting.

2. **User service** – Handles sign-up, login, password resets, profile management and subscription status. Leverages Firebase Authentication for ease of use and multi-provider sign-in (email/password, Google, Facebook). Firebase Authentication is free for most options under the Spark and Blaze plans.
3. **Listing & request service** – Manages skill listings, requests and matching logic. Exposes CRUD endpoints and uses message queues/events to update search indices and notifications.
4. **Match-making service (AI)** – Implements algorithms to match users based on offered and requested services. Initially use rule-based matching (e.g., skill tags, location proximity). Later, train a machine-learning model using historical matches to improve recommendations. Use Python (FastAPI) or Node.js (NestJS) microservice; schedule with Celery (if Python) or BullMQ (if Node). Use vector similarity search for multi-party matching when more than two users have complementary services.
5. **Payment service** – Integrates with Geidea’s payment gateway to handle subscription payments and donations. This service creates payment sessions, verifies transactions, handles callbacks/webhooks, and updates membership status. Geidea provides an online payment gateway in Egypt and KSA enabling businesses to accept online payments securely; merchants can embed Geidea’s payment gateway in websites or apps.
6. **Wishlist/donation service** – Stores wishes created by users in the “Tahadou Tahabou” section. Exposes endpoints to create wishes (membership required) and donate to wishes (no membership required). Supports optional anonymity for donors.
7. **Moderation service** – Filters user-submitted listings/requests using a dictionary of banned keywords and optionally a third-party content moderation API. Flagged content is sent to admins for review. Use Cloud Functions to perform checks on write events.
8. **Identity verification service** – Integrates with KYC providers. For example, **Valify** offers National ID transliteration and a “Root of Trust” that verifies Egyptian national ID numbers through the National Registry. Similarly, **Uqudo** provides document scanning, facial recognition and data validation against national registries. The verification service collects ID photos, sends them to the provider API, and matches the extracted name with the user’s name. This ensures users are real and reduces fraud.
9. **Notification service** – Sends in-app, email and push notifications (Firebase Cloud Messaging) about booking requests, matches and wish updates.
10. **Admin dashboard** – Allows admins to manage users, subscriptions, flagged content, and financial reports.

2.3 Data Storage

Data type	Proposed storage	Reason
User accounts & authentication	Firebase Authentication	Provides out-of-the-box secure authentication; free tier supports unlimited social sign-ins.
User profiles, listings, requests, wishes	Cloud Firestore	Cloud Firestore stores data as documents in collections, supports complex,

Data type	Proposed storage	Reason
		hierarchical data models, indexed queries and automatic scaling. It is multi-region with automatic scaling, 99.999 % uptime and 1 M concurrent connections, making it suitable for high-load scenarios.
Real-time chat & presence	Firebase Realtime Database	Provides ultra-low-latency real-time updates (10 ms) and built-in client presence detection. Use for chat messages and presence status. To avoid scaling limits (200 k connections and 1k writes/s), shard chat rooms across databases if needed.
Payments & transaction logs	Cloud SQL (PostgreSQL)	Payment records require relational integrity and transaction capabilities. Use Cloud SQL or Amazon RDS (PostgreSQL) to store payments, subscription invoices and audit logs. PostgreSQL handles ACID transactions and complex queries.
Files (images, ID photos, documents)	Firebase Storage	Stores profile images, service photos and uploaded IDs. Leverage free quotas for storing small files and pay-as-you-go beyond quotas. Images can be served via CDN.
Search indexing	Algolia or MeiliSearch	Offload search to a dedicated search engine for fast full-text queries. Sync Firestore data to Algolia via Cloud Functions. Algolia's free tier supports 10 k records; MeiliSearch is open-source and can run on your own server.
Analytics	Google Analytics for	Track user behaviour,

Data type	Proposed storage	Reason
	Firebase	conversion funnels and page views. Use GA4 to understand retention and marketing effectiveness.

2.4 Infrastructure & Deployment

- **Environment:** Use **Google Cloud Platform** (GCP). Firebase projects are Google Cloud projects; this allows using Cloud Functions, Cloud Run, Cloud SQL, Firestore and Realtime Database under one billing account.
- **Cloud Functions / Cloud Run:** Host microservices and background tasks as serverless functions. Functions scale automatically and integrate natively with Firebase products.
- **CI/CD:** Use **GitHub Actions** or **GitLab CI** to lint, test and deploy both frontend and backend. Deploy preview branches to Firebase Hosting via the CLI. Use environment variables to separate dev/staging/prod.
- **Infrastructure as Code:** Use **Terraform** to manage Firebase resources, Cloud SQL instances, VPC connectors, secret manager, etc. This ensures repeatable deployments.
- **Monitoring & Logging:** Use **Firebase Crashlytics** for client crash reports, **Cloud Monitoring** for backend logs and metrics, and **Stackdriver Logging** for aggregated logs.
- **Security:** Use **Firebase App Check** to verify calls from legitimate clients, enforce HTTPS, and use Firestore and Realtime Database security rules to restrict data access. Store API keys and secrets in **Secret Manager**.

2.5 Scalability Considerations

- **Automatic scaling:** Cloud Firestore scales automatically to millions of concurrent connections and tens of thousands of writes per second. Realtime Database scales to ~200k connections and 1k writes/s per database; shard data across multiple databases if the chat load increases.
- **Cache & CDN:** Use Firebase Hosting's CDN for static assets and Cloud Firestore's built-in caching. Use Cloud Functions to cache frequently requested queries.
- **Queueing & rate limiting:** Use **Cloud Tasks** or **Pub/Sub** to process heavy tasks (e.g., sending emails, verifying IDs) asynchronously. Rate-limit external API calls (e.g., KYC providers) to stay within their quotas.
- **Regional deployment:** Deploy the Firestore database in a region close to the primary users (e.g., europe-west1 for Egypt). Multi-region support ensures high availability and compliance.

3. Feature Implementation Details

3.1 Sign-Up and Membership

- **Free sign-up:** Users register with email/password or social logins via Firebase Authentication. At this stage they can **browse listings and leave reviews** (guest mode). They cannot create listings or accept requests.

- **Membership subscription:** To **post a listing, request or offer**, the user must purchase a subscription (Basic, Standard, Pro or Business). Use Geidea's payment API to create checkout sessions. Upon successful payment, update the user's membership record in Cloud SQL and Firestore.
- **Subscription plans:** The **Subscription Plan Structure** PDF lists four plans. The Basic plan (30 EGP/3 months or 25 SAR/3 months) includes up to **9 listings, 9 bookings and limited messaging**. The Standard plan (50 EGP/3 mo) offers **12 active listings and 12 bookings**. The Pro plan (80 EGP/3 mo) provides **unlimited listings and bookings** and adds advanced analytics, priority search ranking and a verified badge. The Business plan (600 EGP/3 mo) includes **multi-member teams, business profile & branding, custom categories and dedicated account manager**. These limits will be enforced in the backend.
- **No free trials:** This plan does **not** provide free trial memberships to end users. Users must purchase a subscription immediately to create or accept listings, though they may browse and donate without registering.
- **Billing cycles:** Support 3-month, 6-month and 12-month billing durations as shown in the plan images. Use scheduled functions (Cron) to process renewals and send reminders.
- **Guest reviews:** Allow non-subscribed users to leave reviews on services they experienced. Store reviews in Firestore and moderate them for offensive content.

3.2 Tahadou Tahabou – Wish & Donation Section

- **Creating a wish:** Only users with an active membership can create a wish. They describe their wish, optionally attach an image or video, set a deadline and select a category (education, healthcare, personal needs, etc.). Validate the description and ensure it doesn't contain banned keywords.
- **Displaying wishes:** The home page shows featured wishes. Users can filter by category or location. Use Firestore to query active wishes and update the home page in real time.
- **Donations without sign-up:** Anyone (including guests) can donate to a wish by entering their name (optional) and payment details. Integrate with Geidea to generate a payment link. Optionally, donors can choose to remain anonymous. After payment is confirmed, the donation amount is stored in Cloud SQL and the wish's collected amount is updated in Firestore.
- **Transparency & reporting:** Provide a dashboard to show how much has been collected per wish. Send receipts by email. Enforce donation limits if required by regulations.

3.3 AI-Powered Matchmaking

- **Initial rule-based algorithm:** Match users by comparing offered skill tags and requested skill tags. Include location proximity and availability to suggest the most compatible matches. When three parties have complementary services, create a circular match ($A \rightarrow B \rightarrow C \rightarrow A$) and notify them.
- **Machine learning model:** After collecting enough data (user interactions, ratings, successful exchanges), train a recommendation model to predict match quality. Use a combination of collaborative filtering and content-based filtering. The model can run in a Cloud Run container or Cloud AI Platform.

- **Implementation:** Expose an endpoint `/match` that accepts user ID and returns match suggestions. Use caching to reduce latency. Provide explanation of why users were matched (e.g., “both offer web development and need car detailing”).

3.4 Content Moderation and Banned Keywords

- Maintain a **list of prohibited services** (e.g., illegal activities, adult content, weapons). Use Cloud Functions to intercept listing creation and check text fields against this list. If matched, reject or flag the listing.
- For more advanced moderation, use a third-party API (e.g., Google Cloud Content Moderation) to analyse images and text for policy violations.
- Provide an admin interface to review flagged content and update the banned list.

3.5 Identity Verification

- **KYC provider integration:** Use an API from Valify or Uqudo to perform identity verification. Valify’s “Root of Trust” service verifies Egyptian national IDs by checking the information against the National Registry. Uqudo offers OCR scanning, NFC reading and data validation against national registries. Choose a provider based on cost, accuracy and API availability.
- **Workflow:** When a user reaches the verification step, capture a photo of their national ID and optionally a selfie. Send the ID image to the KYC API; receive extracted name, number and validity status. Compare the extracted name with the user’s account name; if they match, mark the user as verified. Store verification status and reference ID from the provider.
- **Privacy:** Encrypt ID images in Firebase Storage and restrict access. Keep only a hash of the ID number to comply with data protection laws.

3.6 Arabic Language Support

- **Translations:** Store translation strings in JSON files. Use `i18n` frameworks (react-i18next for web and intl packages for Flutter) to load the appropriate language. Provide a language selector.
- **RTL layout:** When the selected language is Arabic, set `<html dir="rtl">` or use the Flutter Directionality widget. The Logto blog notes that text alignment, content direction and scrollbar position must be reversed in RTL layouts. Using the `dir="rtl"` attribute automatically handles these challenges. Use CSS logical properties (`margin-inline-start`, `padding-inline-end`) to handle margins and paddings in both directions. Mirror icons or use CSS transforms to flip icons in RTL. Localise dates and numbers using `toLocaleString` with the `ar-u-nu-arab` locale.
- **Arabic fonts:** Use fonts that support Arabic script (e.g., Cairo, Amiri) and ensure proper line spacing.

3.7 Geidea Payment Integration

- **Subscription payments:** Use Geidea’s payment gateway to create subscriptions. Provide a checkout page where users select a plan and pay via card or digital wallets. After successful payment, handle the callback from Geidea to update the user’s membership record.

- **Donation payments:** For wishes, generate payment links that donors can pay without logging in. Use Geidea's "pay by link" API to accept multiple currencies (EGP, SAR). Provide donors with receipts and update the wish record.
- **Security:** Ensure PCI compliance by redirecting users to Geidea's secure payment page or using tokenised card inputs. Store only tokens and transaction IDs.

3.8 Additional Features & Considerations

- **Messaging and notifications:** Use Firestore or Realtime Database for messaging; send push notifications via Firebase Cloud Messaging for booking requests, matches and wish updates.
- **Analytics and A/B testing:** Use Firebase Analytics and Remote Config to test UI changes (e.g., how many users upgrade to Pro plan).
- **App distribution:** Publish the mobile app to Google Play and Apple App Store. Account fees: Google Play charges a one-time \$25 developer fee, while Apple App Store requires a \$99/year developer program. Budget for these fees.
- **Customer support:** Integrate a chat widget (e.g., Intercom) on the web and add a support screen in the app. Provide FAQ pages.
- **Legal & compliance:** Draft Terms & Conditions, Privacy Policy, Refund Policy and community guidelines. Ensure compliance with Egypt's data protection laws and KSA regulations. Provide a reporting mechanism for abuse.

4. Choosing Firebase Plans

Firebase offers **Spark (free)** and **Blaze (pay-as-you-go)** plans.

- **Development phase (free):** Start on the **Spark plan** to avoid charges. The Spark plan includes full usage of no-cost Firebase products (Authentication, Cloud Messaging, Crashlytics, etc.) and provides a free quota for paid products like Cloud Firestore, Cloud Storage and Realtime Database. For example, you receive 20 k writes/day and 50 k reads/day for Firestore, which is sufficient for development and initial beta testing.
- **Production phase (paid):** When traffic increases, upgrade to the **Blaze plan**. Blaze continues to provide the free quota but charges for usage beyond the quota. It also unlocks Cloud Functions, Cloud Run and other Google Cloud products that are essential for scalability. Evaluate usage regularly with the pricing calculator and set budget alerts.

5. Database Choice Justification

The app requires storing user profiles, listings, messages and payments. There is a debate between NoSQL (Firestore/Realtime) and relational databases. Firestore stores data as documents and collections and supports complex, hierarchical data with automatic scaling. Queries are indexed by default and performance depends on the result set, not dataset size. Realtime Database uses a single JSON tree and offers low-latency presence detection but requires sharding to scale beyond ~200 k connections.

For **listings, requests and wishes**, Firestore is preferred due to its ability to handle complex queries, indexing and automatic scaling. For **real-time chat and presence**, Realtime Database

is ideal. For **payment records**, a relational database (PostgreSQL) ensures ACID transactions and relational integrity.

6. Chosen Tech Stack

To avoid confusion, this chapter summarises the **selected** technology stack for the initial build. In earlier sections we mentioned several alternatives (e.g., React Native vs Flutter, Python vs Node.js). Here we make definitive choices based on the requirements, scalability and ease of development:

- **Web Front-end: React with Next.js.** Next.js provides server-side rendering and SEO benefits while allowing us to build the site as a single-page application. It integrates well with Firebase and supports internationalisation and RTL layouts.
- **Mobile Application: Flutter.** Flutter allows us to build a single codebase that targets both Android and iOS with native-like performance. It has first-class support for right-to-left languages and a rich set of UI widgets. We considered React Native, but Flutter was chosen for its superior performance and widget flexibility.
- **Backend & Architecture: Node.js with TypeScript (NestJS) following a modular microservices approach.** We will design the system as a **modular monolith** built with NestJS, where each domain (user, listing, wish, payment) is implemented as a separate module. NestJS's built-in dependency injection makes it easy to break modules into independent microservices later if needed. For event-driven tasks (e.g., sending emails, syncing search indices, moderation) we will use **serverless functions (Cloud Functions)**, which are lightweight, scalable and integrate with Firebase. The AI match-making service will be a **stand-alone microservice written in Python with FastAPI** because Python's data-science libraries are better suited for recommendation algorithms. This hybrid architecture (modular monolith + serverless functions + separate AI microservice) gives us the flexibility and scalability of microservices without the complexity of fully distributed systems.
- **Database:** Use a **polyglot persistence** approach:
- **Cloud Firestore** for core data (users, listings, requests, wishes) due to its scalability and query capabilities.
- **Firebase Realtime Database** for chat and presence due to its ultra-low latency.
- **PostgreSQL (Cloud SQL)** for financial transactions and payment logs where ACID transactions are critical.
- **Authentication: Firebase Authentication** for user sign-in with email/password and social logins.
- **File Storage: Firebase Storage** for images and documents.
- **APIs & Services:**
- **Geidea** for payment processing (subscriptions and donations).

- **Valify (Egypt)** and **Uqudo (MENA)** for identity verification.
- **Algolia** (or self-hosted MeiliSearch) for search indexing.
- **DevOps & Hosting:**
- **Firebase Hosting** and **Vercel** for web front-end hosting.
- **Cloud Functions** and **Cloud Run** for backend services.
- **Terraform** for infrastructure as code and **GitHub Actions** for CI/CD.
- **Design Patterns & Methodologies:**
- Use **Model–View–Controller (MVC)** within NestJS modules to separate concerns between controllers (HTTP handlers), services (business logic) and models (data transfer objects).
- Apply **Domain-Driven Design (DDD)** to model core domains (User, Listing, Wish, Payment) with entities, value objects and repositories. This ensures that business rules are encapsulated and easy to evolve.
- Use **Event-Driven Architecture** for decoupling cross-cutting concerns such as notifications and analytics; events will be emitted via message queues (Pub/Sub) and processed by Cloud Functions.

These choices balance developer productivity, cost and scalability. Optional technologies mentioned earlier, such as React Native or additional databases, can be revisited in future versions if business requirements change.

7. Development Timeline (2–3 Months)

The timeline assumes a small dedicated team (backend developer, frontend developer, mobile developer, QA and project manager). Tasks may overlap. Because the founders requested a **2–3 month** schedule, the plan allocates up to 12 weeks for development and testing.

Phase	Duration	Activities
Phase 1: Requirements & Architecture	Week 1	Finalise requirements with stakeholders. Define user stories, acceptance criteria and UI wireframes. Confirm the selected tech stack, data model, architectural patterns and design patterns as specified in this plan (e.g., modular microservices with NestJS, MVC/DDD, serverless functions). The founders do not need to choose these;

Phase	Duration	Activities
Phase 2: Core Backend & Authentication	Weeks 2-4	they are already defined. Set up repositories, CI/CD and Firebase project.
		Implement authentication (Firebase Auth) without offering free trials for end users. Build user profile service and subscription logic. Create Firestore collections for users, listings and requests. Integrate Geidea sandbox for payments. Implement membership gating and enforce plan limits. Start building moderation service and banned keyword checks. Set up PostgreSQL (Cloud SQL) for payments.
Phase 3: Front-end Development	Weeks 3-6	Build the web UI with Next.js using the Firebase AI-generated pages as a starting point. Implement pages: home, listings, listing details, create listing, requests, wish/donation page, profile and admin dashboard. Add Arabic translations and RTL support. Build mobile app screens simultaneously using Flutter/React Native. Connect to backend APIs.
Phase 4: Matching & Messaging	Weeks 5-7	Develop the rule-based matchmaking service and chat module using Realtime Database. Implement notifications (email, in-app, push). Add search functionality using Algolia/MeiliSearch and integrate filters. Continue improving UI/UX and add analytics.

Phase	Duration	Activities
Phase 5: KYC & Verification Integration	Weeks 6–8	Integrate Valify/Uqudo API for ID verification. Implement verification flow in the app (upload ID and selfie). Store verification status and handle failures. Provide admin documentation.
Phase 6: Wish/Donation Module & Final Polishing	Weeks 7–9	Build the wish creation flow with membership check, donations (anonymous or not), progress bar and receipts. Finalise moderation and admin tools. Conduct thorough QA testing on web and mobile. Localise all strings and refine RTL layouts.
Phase 7: Beta Launch & Iteration	Weeks 10–12	Deploy to staging on the Spark plan. Invite beta testers. Monitor performance, fix bugs, adjust pricing plan enforcement and refine matching algorithms. Prepare marketing materials. When ready, upgrade to Blaze plan and launch to production.

Total estimated duration: **10–12 weeks** (~2–3 months). A minimal viable product (MVP) could be launched by **week 8** (roughly 2 months) if scope is reduced (e.g., postpone machine-learning-based matching and advanced analytics).

8. Future Enhancements

- **Machine-learning match making** – After launching, collect data to train better recommendation models.
- **Gamification** – Introduce badges and leaderboards to incentivise participation.
- **Marketplace add-ons** – Allow users to purchase extra bookings or listings beyond their plan limits.
- **Event management** – For the Business plan, allow companies to create events or workshops and accept registrations.
- **API for third-party integration** – Provide a public API for businesses or other apps to integrate with SkillSwap.
- **Global expansion** – Add multi-currency support and additional languages.

9. Conclusion

This technical plan provides a **human-friendly, comprehensive guide** to building the SkillSwap platform. By leveraging Firebase's free tier in the development phase and upgrading to pay-as-you-go services in production, using Firestore for scalable data storage and integrating third-party identity verification and payment services, we can deliver a robust marketplace that can handle high user loads without crashing. The plan emphasises modular architecture, strong security practices, RTL support and a phased timeline to ensure on-time delivery and future scalability.

10. APIs and Pricing Overview

The platform relies on several external services. Below is an indicative list of APIs and their typical pricing models. Exact prices may change; contact each provider for current rates.

API/service	Purpose	Availability in Egypt/Saudi	Pricing (approx.)	Notes
Firebase Authentication	User sign-in (email/password, social logins)	Global	Spark plan is free for unlimited email/social logins. Phone-number authentication charges per SMS (varies by country).	Good for quick onboarding; no free trials offered to end users.
Cloud Firestore	Stores users, listings, requests, wishes	Global (multi-region)	Free quota (~1 GiB storage, 20 k writes/day, 50 k reads/day on Spark). Beyond quota, charges apply per read/write and per GiB stored (Blaze plan).	Automatically scales to 1 M concurrent connections.
Realtime Database	Handles chat and presence	Global	Free quota (~1 GiB storage, 1 GB downloads/day on Spark). Additional reads/writes billed on Blaze.	Ultra-low latency; scales to ~200k connections per database.

API/service	Purpose	Availability in Egypt/Saudi	Pricing (approx.)	Notes
Cloud Functions / Cloud Run	Runs serverless backend code	Global	You pay for compute time and invocations on Blaze. A free tier of 2 M invocations/month exists.	Used for API gateway, matchmaking and background tasks.
Cloud SQL (PostgreSQL)	Stores payments and transactions	Regional	Pricing depends on instance size; entry-level shared core instance costs about US\$7–10/month . Storage billed separately.	Required for ACID transactions (subscriptions, donations).
Firebase Storage	Stores images and ID documents	Global	Free 5 GiB on Spark; pay per GiB stored and data egress on Blaze.	Use rules to restrict access.
Algolia / MeiliSearch	Search indexing	Algolia is global; MeiliSearch can be self-hosted anywhere	Algolia has a free tier (10 k records, 100 k operations/month). Paid plans start at ~US\$1 per 1k search requests . MeiliSearch is open source (no licence cost) but requires a server (~US\$10/month).	Improves search experience.
Geidea Payment Gateway	Processes subscription and donation payments	Available in Egypt and Saudi Arabia	Geidea charges a transaction fee per payment (exact fee depends on agreement and payment method).	Supports multiple currencies (EGP, SAR) and payment methods; merchants must sign a contract.

API/service	Purpose	Availability in Egypt/Saudi	Pricing (approx.)	Notes
			Typically a small percentage (e.g., 2–3 %) plus a fixed fee.	
Valify (Root of Trust service)	Verifies Egyptian national IDs by checking against the National Registry	Operates in Egypt	Pricing is negotiated per verification with volume discounts. Public information is scarce; expect per-verification fees (~US\$0.50–1.50) plus possible setup fees.	Suitable for Egyptian users; contact Valify for a quote.
Uqodo	Provides e-KYC, OCR, NFC reading, face verification and data validation against national registries	Operates across the Middle East & Africa, including Egypt and Saudi Arabia	Uqodo's pricing is customised based on features and geography; the FAQ states that pricing varies and discounts apply for larger volumes. Expect per-verification fees similar to other providers (approx. US\$0.50–1.50).	Offers both KYC and KYB; ideal for verifying users in multiple countries.
Third-party content moderation (optional)	Detects inappropriate content in listings and images	Global	Services like Google Cloud Content Safety API charge roughly US\$1.50 per 1k text blocks or images .	Helps enforce banned-services policy.

Notes:

- Both **Valify** and **Uqudo** support operations in Egypt and the Gulf region. Valify specialises in Egyptian ID verification, while Uqudo operates across the Middle East and Africa. Pricing is typically per verification and negotiated directly with the provider. Contact sales to obtain a tailored quote.
- For small verification volumes in development, consider using a free test environment provided by the KYC provider. For production, budget for at least a few hundred verifications per month.
- This plan does **not** offer free trials to end users. Users must purchase a membership before creating or accepting listings. Guests may browse content, leave reviews or donate without registering where applicable.